Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

по Лабораторной работе № 6

«Работа с БД в СУБД MongoDB»

по дисциплине «Проектирование и реализация баз данных»

Обучающиеся Дедкова Анастасия Викторовна
Факультет прикладной информатики
Группа K3240
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2023
Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2024/2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ..... 3
1 Выполнение (Часть 1).....5
1.1 Проверка работоспособности.....5
1.2 Манипуляции с БД.....6
2 Выполнение (Часть 2)..... 9
2.1 Создание и заполнение БД.....9
2.2 Запросы к БД..... 10
2.3 Модифицирование запроса..... 12
2.4 Запрос 1.....13
2.5 Запрос 2.....13
2.6 Запрос 3.....14
2.7 Запрос 4.....15
2.8 Запрос 5.....16
2.9 Запрос 6.....17
3 Выполнение (Часть 3)..... 19
3.1 Практическое задание 3.1.1..... 19
3.2 Практическое задание 3.1.2..... 21
3.3 Практическое задание 3.2.1..... 22
3.4 Практическое задание 3.2.2..... 23
3.5 Практическое задание 3.2.3..... 24
3.6 Практическое задание 3.3.1:..... 24
3.7 Практическое задание 3.3.2..... 26
3.8 Практическое задание 3.3.3..... 26
3.9 Практическое задание 3.3.4..... 27
3.10 Практическое задание 3.3.5..... 28
3.11 Практическое задание 3.3.6.....29
3.12 Практическое задание 3.3.7..... 30
3.13 Практическое задание 3.4.1..... 31
4 Выполнение (Часть 4)..... 33
4.1 Практическое задание 4.1.1..... 35
4.2 Практическое задание 4.2.1..... 37
4.3 Практическое задание 4.3.1..... 38
4.4 Практическое задание 4.4.1..... 40
ЗАКЛЮЧЕНИЕ.....43

ВВЕДЕНИЕ

Цель работы – овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Практическое задание:

Часть 1:

- Установите MongoDB.
- Проверьте работоспособность системы запуском клиента mongo.
- Выполните методы:
  - db.help()
  - db.help
  - db.stats()
- Создайте БД learn.
- Получите список доступных БД.
- СОздайте коллекцию unicorns вставив в неё документ {name: ‘Aurora’, gender:’f’, weight:450}.
- Просмотрите список текущих коллекций.
- Переименуйте коллекцию unicorns.
- Просмотрите статистику коллекции.
- Удалите коллекцию.
- Удалите БД learn.

Часть **2.1.1**:

- Создайте базу данных learn.
- Заполните коллекцию единорогов unicorns
- Используя второй способ, вставьте в коллекцию единорогов документ.
- Проверьте содержимое коллекции с помощью метода find.

Часть **2.2.1**:

- Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.
- Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций findOne и limit.

Часть **2.2.2**:

- Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

Часть **2.2.3**:

- Вывести список единорогов в обратном порядке добавления.

Часть **2.2.4:**

- Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

Часть **2.3.1**:

- Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

Часть **2.3.2**:

- Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

Часть **2.3.3**:

- Найти всех единорогов, не имеющих ключ vampires.

Часть **2.3.4**:

- Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

1 Выполнение (Часть 1)

1.1 Проверка работоспособности

Выполним методы:

a) db.help()

Метод db.help() в MongoDB shell (mongosh) используется для вывода справочной информации о доступных методах и командах, которые можно выполнять с текущей базой данных (db) (Рисунок 1).

```
> db.help()
< Database Class

  getMongo              Returns the current database connection
  getName               Returns the name of the DB
  getCollectionNames    Returns an array containing the names of all collections in the current
                        database.
  getCollectionInfos    Returns an array of documents with collection information, i.e.
                        collection name and options, for the current database.
  runCommand            Runs an arbitrary command on the database.
  adminCommand          Runs an arbitrary command against the admin database.
  aggregate             Runs a specified admin/diagnostic pipeline which does not require an
                        underlying collection.
```

Рисунок 1 – Результат выполнения метода db.help()

b) db.help

В mongosh вывод db.help и db.help() выглядит одинаково, потому что оболочка автоматически выполняет функцию при доступе, если это возможно (рисунок 2).

```
> db.help
< Database Class

  getMongo              Returns the current database connection
  getName               Returns the name of the DB
  getCollectionNames    Returns an array containing the names of all collections in the current
                        database.
  getCollectionInfos    Returns an array of documents with collection information, i.e.
                        collection name and options, for the current database.
  runCommand            Runs an arbitrary command on the database.
  adminCommand          Runs an arbitrary command against the admin database.
  aggregate             Runs a specified admin/diagnostic pipeline which does not require an
                        underlying collection.
  getSiblingDB          Returns another database without modifying the db variable in the shell
                        environment.
  getCollection         Returns a collection or a view object that is functionally equivalent
                        to using the db.<collectionName>.
  dropDatabase          Removes the current database, deleting the associated data files.
  createUser            Creates a new user for the database on which the method is run.
                        db.createUser() returns a duplicate user error if the user already
                        exists on the database.
```

Рисунок 2 – Результат выполнения метода db.help

c) db.stats()

В MongoDB используется для получения статистической информации о текущей базе данных (рисунок 3).



Рисунок 3 – Результат выполнения метода db.stats()

1.2 Манипуляции с БД

Создали БД learn и создали коллекцию unicorns вставив в неё документ {name: ‘Aurora’, gender:’f’, weight:450}. (рисунок 4, 5, 6).

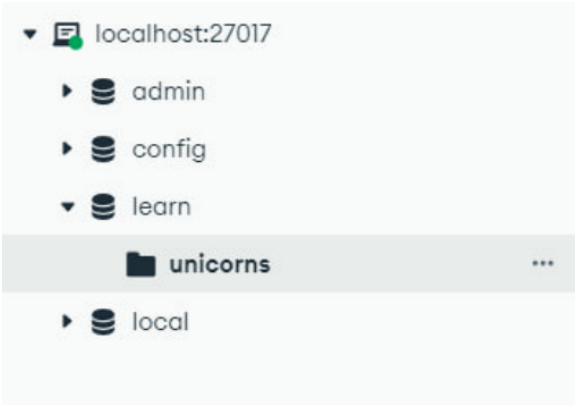


Рисунок 4 – Создание базы

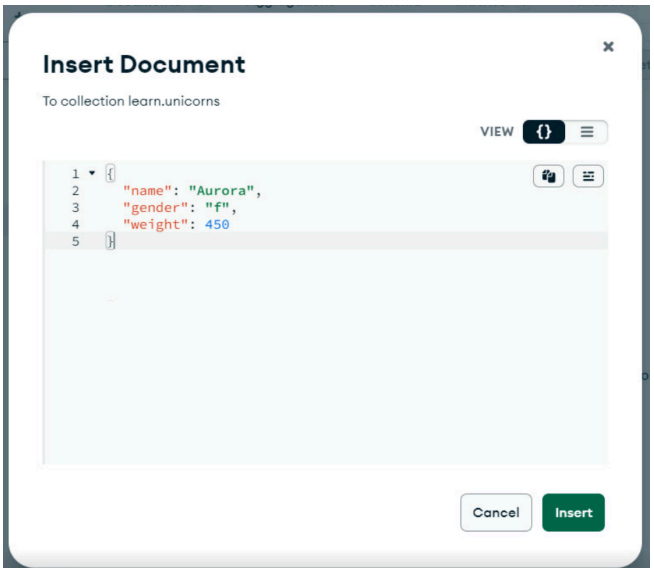
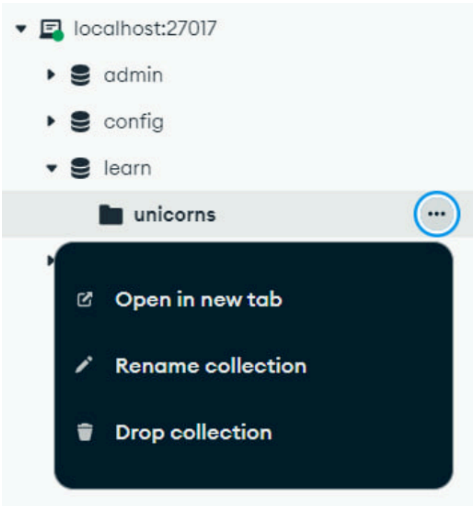


Рисунок 5 – Создание коллекции и документа



Рисунок 6 – Созданный документ

Переименовали коллекцию unicorns (рисунок 7, 8).



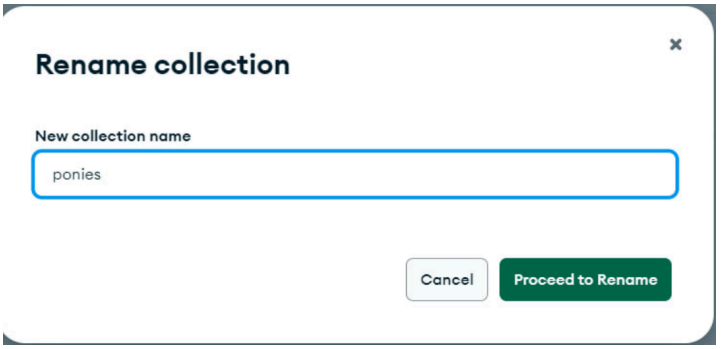


Рисунок 7, 8 – Переименование коллекции

Статистику коллекции можно увидеть на рисунке 9.



Рисунок 9 – Статистика коллекции

Удалим коллекцию (рисунок 10).

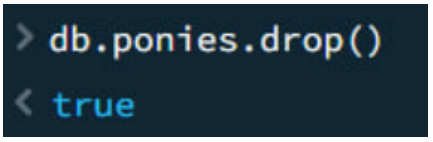
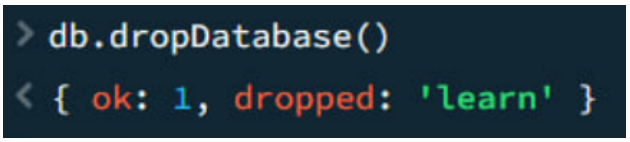


Рисунок 10 – Удаление коллекции

Удалим БД learn (рисунок 11, 12).



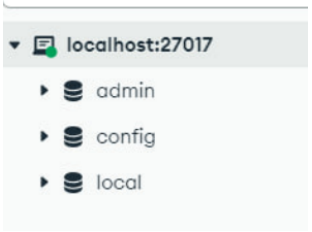


Рисунок 11, 12 – Удаление БД

2 Выполнение (Часть 2)

2.1 Создание и заполнение БД

Создали базу данных learn и заполнили коллекцию единорогов unicorns (рисунок 13).

```
> db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
  db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
  db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
  db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
  db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
  db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
  db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690,  gender: 'm', vampires: 39});
  db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
  db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
  db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
  db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
< DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6832faa0560306867137ab97')
    }
  }
learn>
```

Рисунок 13 – Создание бд и заполнение коллекции способ 1

Используя второй способ, вставили в коллекцию единорогов новый документ (рисунок 14).

```
> document = ({
    name: "Dunx",
    loves: ["grape", "watermelon"],
    weight: 704,
    gender: "m",
    vampires: 165
  })
  db.unicorns.insert(document)
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6832fb47560306867137ab98')
    }
  }
learn>
```

Рисунок 14 – Вставка нового документа в коллекцию вторым способом

Проверили содержимое коллекции с помощью метода find (рисунок 15).



Рисунок 15 – Проверка содержимого коллекции

2.2 Запросы к БД

Сформируйте запросы для вывода списков самцов и самок единорогов.
db.unicorns.find({ gender: "m" }).sort({ name: 1 })

На рисунке 16 результат выполнения запроса.



Рисунок 16 – Результат выполнения запроса

Ограничьте список самок первыми тремя особями.

db.unicorns.find({ gender: "f" }).sort({ name: 1 }).limit(3)

На рисунке 17 результат выполнения запроса.



Рисунок 17 – Результат выполнения запроса

Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций findOne и limit.

db.unicorns.find({ gender: "f", loves: "carrot" }).limit(1)

На рисунке 18 результат выполнения запроса.



Рисунок 18 – Результат выполнения запроса

2.3 Модифицирование запроса

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
db.unicorns.find(
  { gender: "m" },
  { loves: 0, _id: 0 }
).sort({ name: 1 }
```

На рисунке 19 результат выполнения запроса.



Рисунок 19 – Результат выполнения запроса

2.4 Запрос 1

Вывести список единорогов в обратном порядке добавления.

```
db.unicorns.find().sort({ $natural: -1 })
```

На рисунке 20 результат выполнения запроса.



Рисунок 20 – Результат выполнения запроса

2.5 Запрос 2

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
db.unicorns.find(
  {},
  {
   name: 1,
   loves: { $slice: 1 },
   _id: 0
  }
)
```

На рисунке 21 результат выполнения запроса.



Рисунок 21 – Результат выполнения запроса

2.6 Запрос 3

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
db.unicorns.find(
  {
    gender: "f",
    weight: { $gte: 500, $lte: 700 }
  },
  {
    _id: 0
  }
)
```

На рисунке 22 результат выполнения запроса.



Рисунок 22 – Результат выполнения запроса

2.7 Запрос 4

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
db.unicorns.find(
  {
   gender: "m",
   weight: { $gte: 500 },
   loves: { $all: ["grape", "lemon"] }
  },
```

```
  {
    _id: 0
  }
)
```

На рисунке 23 результат запроса.



Рисунок 23 – Результат выполнения запроса

2.8 Запрос 5

Найти всех единорогов, не имеющих ключ vampires.

```
db.unicorns.find(
  { vampires: { $exists: false } }
)
```

На рисунке 24 результат запроса.



Рисунок 24 – Результат выполнения запроса

2.9 Запрос 6

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
db.unicorns.find(
  { gender: "m" },
  {
   _id: 0,
   name: 1,
   loves: { $slice: 1 }
  }
).sort({ name: 1 })
```

На рисунке 25 результат выполнения запроса.



Рисунок 25 – Результат выполнения запроса

3 Выполнение (Часть 3)

3.1 Практическое задание 3.1.1

Создали коллекцию towns (рисунок 26).

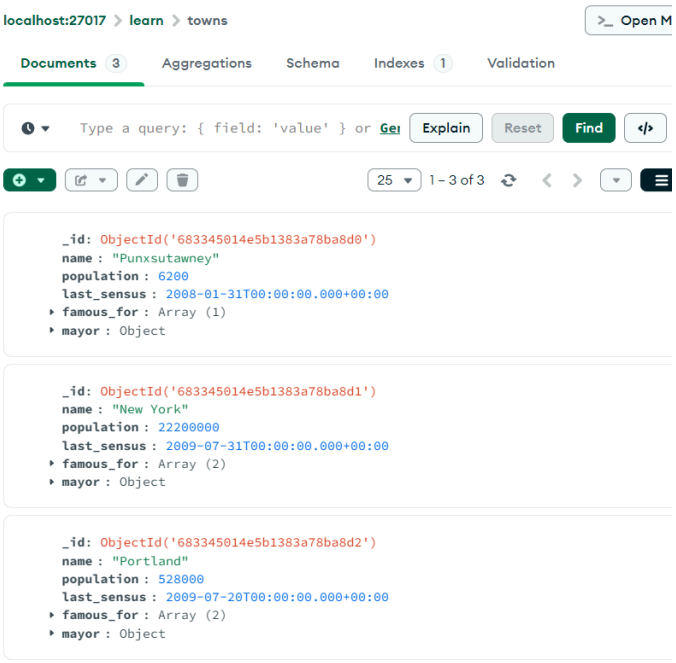


Рисунок 26 – Созданная коллекция towns

Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

db.towns.find({ "mayor.party": "I" }, {_id: 0, name: 1, mayor: 1})

На рисунке 27 результат выполнения запроса.



Рисунок 27 – Результат выполнения запроса

Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

```
db.towns.find(
  { "mayor.party": { $exists: false } },
  {
    _id: 0,
    name: 1,
    mayor: 1
  }
)
```

На рисунке 28 результат выполнения запроса.

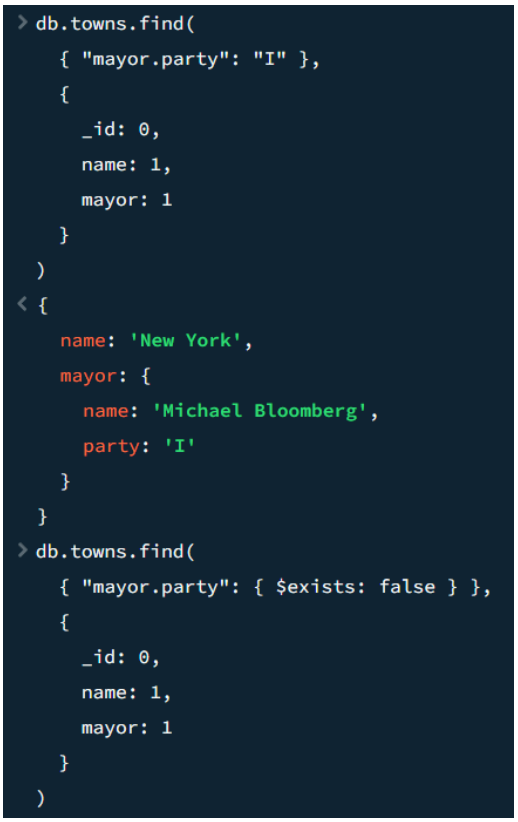


Рисунок 28 – Результат выполнения запроса.

3.2 Практическое задание 3.1.2

Сформировать функцию для вывода списка самцов единорогов.

```
function showMaleUnicorns() {
  var cursor = db.unicorns.find({ gender: "m" }); null;
  cursor.forEach(function(obj) {
    print(obj.name);
  });
}
showMaleUnicorns()
```

На рисунке 29 результат вызова функции.



Рисунок 29 – Результат вызова функции.

Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.

Вывести результат, используя forEach.

```
var cursor = db.unicorns.find({ gender: "m" }); null;
cursor.sort({ name: 1 }).limit(2); null;
cursor.forEach(function(obj) {
  print(obj.name);
});
```

На рисунке 30 результат курсора.

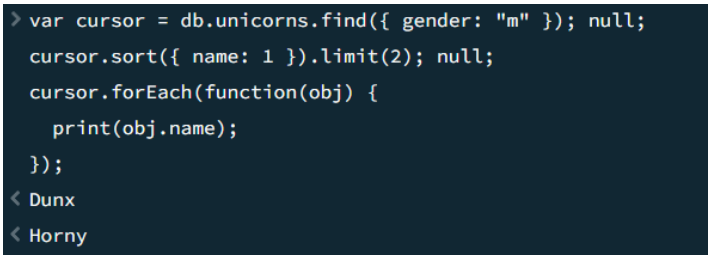


Рисунок 30 – Результат выполнения курсора.

3.3 Практическое задание 3.2.1

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
db.unicorns.countDocuments({
  gender: "f",
  weight: { $gte: 500, $lte: 600 }
```

})

На рисунке 31 результат выполнения запроса.

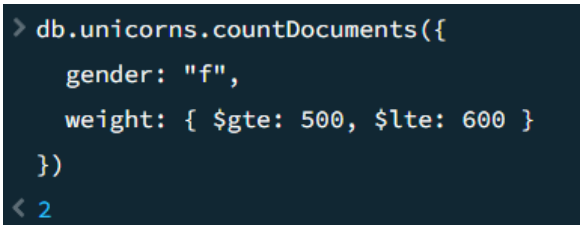


Рисунок 31 – Результат выполнения запроса.

3.4 Практическое задание 3.2.2

Вывести список предпочтений.

```
db.unicorns.aggregate([
  { $unwind: "$loves" },
  { $group: { _id: "$loves" } }
])
```

На рисунке 32 результат выполнения запроса.

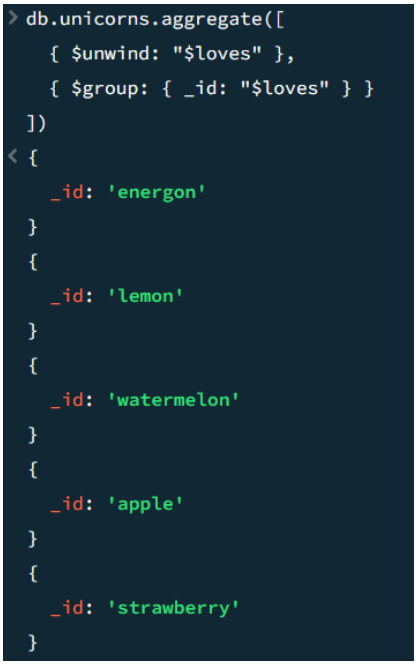


Рисунок 32 – Результат выполнения запроса

3.5 Практическое задание 3.2.3

Подсчитать количество особей единорогов обоих полов.

```
db.unicorns.aggregate([
  { $group: { _id: "$gender", count: { $sum: 1 } } }
])
```

На рисунке 33 результат запроса.

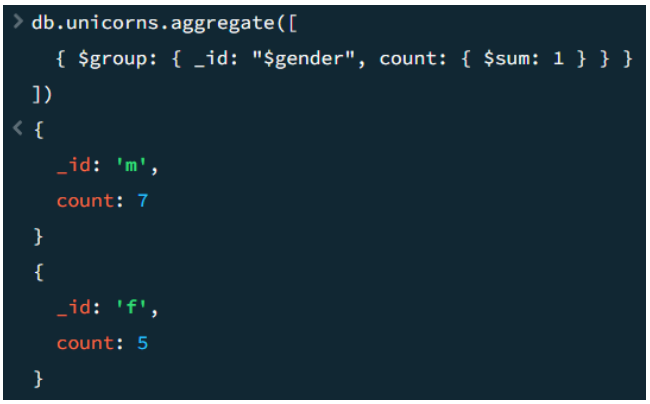


Рисунок 33 – Результат выполнения запроса

3.6 Практическое задание 3.3.1:

Выполнить команду:

```
> db.unicorns.save({name: 'Barny', loves: ['grape'],
weight: 340, gender: 'm'})
```

На рисунке 34 результат выполнения команды.

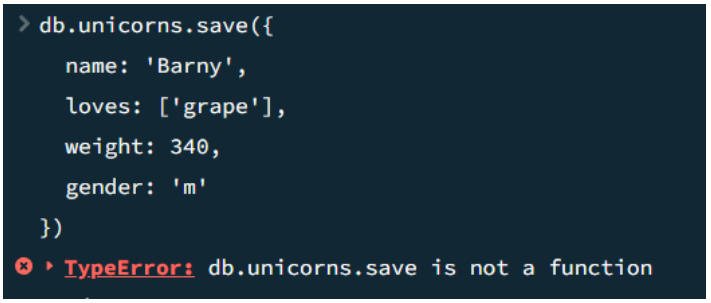


Рисунок 34 – Результат выполнения команды

В mongosh метод save() больше не поддерживается. Можно сделать по другому.

Использовать insertOne()

```
db.unicorns.insertOne({
  name: 'Barny',
  loves: ['grape'],
  weight: 340,
  gender: 'm'
})
```

На рисунке 35 результат выполнения команды.

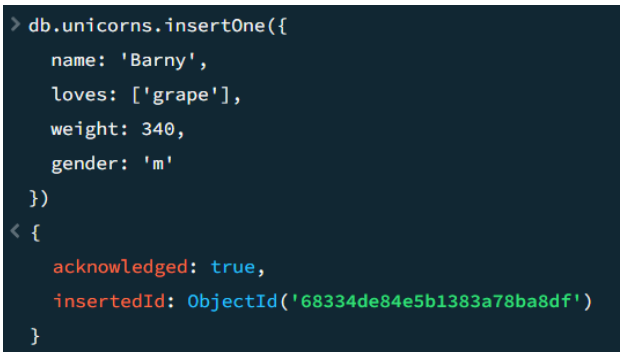


Рисунок 35 – Результат выполнения команды.

Проверить содержимое коллекции unicorns.

```
db.unicorns.find({ name: 'Barny' }).pretty()
```

На рисунке 36 результат выполнения команды.



Рисунок 36 – Результат выполнения команды

3.7 Практическое задание 3.3.2

Для самки единорога Ayna внести изменения в БД: теперь ее вес 800, она убила 51 вапмира.

```
db.unicorns.updateOne(
  { name: "Ayna" },
  { $set: { weight: 800, vampires: 51 } }
)
```

Проверить содержимое коллекции unicorns.

```
db.unicorns.find({ name: "Ayna" }).pretty()
```

На рисунке 37 результат выполнения команд.



Рисунок 37 – Результат выполнения команд

3.8 Практическое задание 3.3.3

Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.

```
db.unicorns.updateOne(
  { name: "Raleigh" },
  { $push: { loves: "redbull" } }
)
```

Проверить содержимое коллекции unicorns.

```
db.unicorns.find({ name: "Raleigh" }).pretty()
```

На рисунке 38 результат выполнения команд.



Рисунок 38 – Результат выполнения команд

3.9 Практическое задание 3.3.4

Всем самцам единорогов увеличить количество убитых вапмиров на 5.

```
db.unicorns.updateMany(
  { gender: "m" },
  { $inc: { vampires: 5 } }
```

)

Проверить содержимое коллекции unicorns.

db.unicorns.find({ gender: "m" }).pretty()

На рисунке 39, 40 до/после выполнения команды.



Рисунок 39, 40 – Результат выполнения команды

3.10 Практическое задание 3.3.5

Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.

db.towns.updateOne(

{ name: "Portland" },

{ $unset: { "mayor.party": "" } }

)

Проверить содержимое коллекции towns.

db.towns.find({ name: "Portland" }).pretty()

На рисунке 41 результат выполнения команд.



Рисунок 41 – Результат выполнения команд

3.11 Практическое задание 3.3.6

Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.

```
db.unicorns.updateOne(
  { name: "Pilot" },
  { $push: { loves: "шоколад" } }
)
```

Проверить содержимое коллекции unicorns.

```
db.unicorns.find({ name: "Pilot" }).pretty()
```

На рисунке 42 результат выполнения команд.



Рисунок 42 – Результат выполнения команд

3.12 Практическое задание 3.3.7

Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.

```
db.unicorns.updateOne(
  { name: "Aurora" },
  { $push: { loves: { $each: ["сахар", "лимоны"] } } }
)
```

Проверить содержимое коллекции unicorns.

```
db.unicorns.find({ name: "Aurora" }).pretty()
```

На рисунке 43 результат выполнения команд.



Рисунок 43 – Результат выполнения команд

3.13 Практическое задание 3.4.1

Создайте коллекцию towns. Удалите документы с беспартийными мэрами.

db.towns.deleteMany({

"mayor.party": { $exists: false }

})

Проверьте содержание коллекции.

db.towns.find().pretty()

На рисунке 44 результат выполнения команд.



Рисунок 44 – Результат выполнения команд

Очистите коллекцию.
db.towns.deleteMany({})
Просмотрите список доступных коллекций.
show collections
На рисунке 45 результат выполнения команд.



Рисунок 45 – Результат выполнения команд

4 Выполнение (Часть 4)

Автоматическое связывание

1 способ. Используя функциональность DBRef, можно установить автоматическое связывание между документами.

Пример применение данной функциональности: добавить новый документ в коллекцию companies:

> apple=({"name" : "apple", "year": 1976})

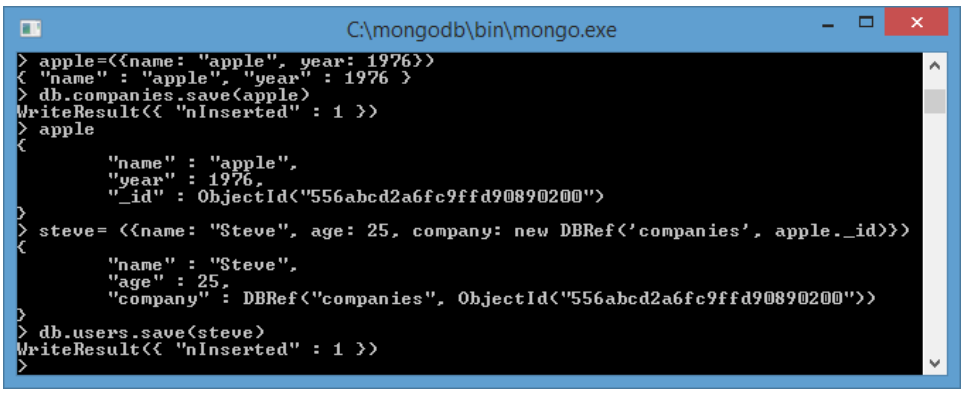
> db.companies.save(apple)

В данном случае сохранение идет с помощью метода save, не insert. Метод save при добавлении нового документа генерирует _id. После сохранения можно вывести документ на консоль: > apple

Далее создать новый документ для коллекции person, у которого ключ company свяжем с только что добавленным документом apple:

> steve = ({"name": "Steve", "age": 25, company: new DBRef('companies', apple._id)})

> db.users.save(steve)



Протестировать:

> db.companies.findOne({_id: steve.company.$id})



Посмотрев на примере, можно разобрать организацию ссылок между документами. Для связывания с документом apple использовалось следующее выражение company: new DBRef('companies', apple._id). Формальный синтаксис DBRef следующий:

{ "$ref" : название_коллекции, "$id": значение [, "$db" : название_бд]}

При тестировании в качестве запроса на выборку указывается выражение _id: steve.company.$id. Так как person.company представляет теперь объект new DBRef('companies', apple._id), то нужно конкретизировать параметр steve.company.$id.

2 способ. Пусть пользователя нужно связать со страной.

Используется коллекция countries, хранящая в качестве идентификатора аббревиатуру и наименование, например:

> db.countries.insert({_id:"us", name:"US"})

Необходимо добавить в коллекцию пользователей ссылку на страну:

>db.users.update({_id:ObjectId("573d7468bfe2c1a9875562e6")},{$set:
{country:{$ref:"countries", $id: "us"}}})

Теперь можно извлечь из коллекции данные о Томе:

>var
tom=db.users.findOne({"_id":ObjectId("573d7468bfe2c1a9875562e6")})

Можно также получить сведения о стране, в которой находится пользователь, опросив коллекцию стран, передав сохраненный ранее идентификатор $id:

> db.countries.findOne({_id:tom.country.$id})

Можно непосредственно запросить у документа о пользователе имя коллекции, хранящейся в поле ссылки:

db[tom.country.$ref].findOne({_id:tom.country.$id})

Последние два запроса эквивалентны, но второй в большей степени управляется данными.

4.1 Практическое задание 4.1.1

Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

```
db.habitats.insertMany([
  {
   _id: "forest",
   name: "Thicket of Fireflies and Sources",
     description: "Thick forests with ancient trees, where the sun's rays break through the fog, hide crystal springs and flowering meadows. Here, among the whisper of leaves and the light of fireflies, unicorns find food, peace and magical shelter."
  },
  {
   _id: "meadow",
   name: "Rainbow Wind Meadows",
```

description: "Open, sunlit meadows dotted with bright flowers and silky grass. Here, under the singing of larks and the trembling of the wind, unicorns play carefree and rest in complete peace."

},

{

_id: "mountain",

name: "Rocks of Ancient Power",

description: "High rocky peaks shrouded in clouds and cool alpine meadows stretch above the world. In these harsh but majestic places live the most hardy and powerful unicorns, accustomed to the winds and heights."

},

{

_id: "cave",

name: "Underground Halls of Tranquility",

description: "Quiet, hidden caves with shimmering crystals and underground streams. Here, unicorns find solitude, rest in silence, or take refuge from the storms of the outside world."

}

])

Включите для нескольких единорогов в документы ссылку на зону обитания, использую второй способ автоматического связывания.

```
db.unicorns.updateMany(
  { name: { $in: ['Horny', 'Aurora'] } },
  { $set: { habitat: { $ref: "habitats", $id: "forest" } } }
)

db.unicorns.updateMany(
  { name: { $in: ['Kenny', 'Ayna'] } },
  { $set: { habitat: { $ref: "habitats", $id: "meadow" } } }
)

db.unicorns.updateMany(
```

```
  { name: { $in: ['Unicrom', 'Dunx'] } },
  { $set: { habitat: { $ref: "habitats", $id: "mountain" } } }
)

db.unicorns.updateOne(
  { name: 'Nimue' },
  { $set: { habitat: { $ref: "habitats", $id: "cave" } } }
)
```

Проверьте содержание коллекции единорогов.

```
db.unicorns.find({ habitat: { $exists: true } }).pretty()
```

На рисунке 46 результат выполнения команд.



Рисунок 46 – Результат выполнения команд

4.2 Практическое задание 4.2.1

Проверьте, можно ли задать для коллекции unicorns индекс для ключа name с флагом unique.

Убедимся, что имена уникальны.

```
db.unicorns.aggregate([
  { $group: { _id: "$name", count: { $sum: 1 } } },
  { $match: { count: { $gt: 1 } } }
])
```

Если результат пустой — все имена уникальны, и можно создавать уникальный индекс.

На рисунке 47 результат выполнения команды.

```
> db.unicorns.aggregate([
    { $group: { _id: "$name", count: { $sum: 1 } } },
    { $match: { count: { $gt: 1 } } }
  ])
<
```

Рисунок 47 – Результат выполнения команды

Создали уникальные индексы

db.unicorns.ensureIndex({ name: 1 }, { unique: true })

На рисунке 48,49 результат выполнения команды.

```
> db.unicorns.ensureIndex({ name: 1 }, { unique: true })
< [ 'name_1' ]
```

Documents 12 | Aggregations | Schema | Indexes 2 | Validation

Create Index | Refresh | VIEWING | INDEXES | SEARCH INDEXES

Name & Definition	Type	Size	Usage	Properties	Status
_id_	REGULAR	36.9 KB	6 (since Sun May 25 2025)	UNIQUE	READY

Рисунок 48, 49 – Результат выполнения команды

4.3 Практическое задание 4.3.1

Получите информацию о всех индексах коллекции unicorns.

db.unicorns.getIndexes()

На рисунке 50 результат выполнения команды.

```
> db.unicorns.getIndexes()
< [
    { v: 2, key: { _id: 1 }, name: '_id_' },
    { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
  ]
```

Рисунок 50 – Результат выполнения команды

- key — поле(я), по которым построен индекс;
- name — идентификатор индекса (используется при удалении);
- unique: true — признак уникальности (если установлен);

Удалите все индексы, кроме индекса для идентификатора.

db.unicorns.dropIndex("name_1")

db.unicorns.getIndexes()

На рисунке 51 результат выполнения команд.

```
> db.unicorns.dropIndex("name_1")
< { nIndexesWas: 2, ok: 1 }
> db.unicorns.getIndexes()
< [ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

Рисунок 51 – Результат выполнения команд

Попытайтесь удалить индекс для идентификатора.

db.unicorns.dropIndex("_id_")

На рисунке 52 результат команды.

```
> db.unicorns.dropIndex("_id_")
✖ ▸ MongoServerError[InvalidOptions]: cannot drop _id index
```

Рисунок 52 – Результат выполнения команды

Он создаётся автоматически при создании коллекции.

Он обязателен — используется для уникальной идентификации каждого документа.

Любая попытка удалить его завершится ошибкой.

4.4 Практическое задание 4.4.1

Создайте объемную коллекцию numbers, задействовав курсор:

for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}

Выберите последних четыре документа.

db.numbers.find().sort({ value: -1 }).limit(4)

На рисунке 53 результат выполнения команд.



Рисунок 53 – Результат выполнения команд

Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра executionTimeMillis)

db.numbers.explain("executionStats").find().sort({ value: -1 }).limit(4)

На рисунке 54 результат выполнения команд.



Рисунок 54 – Результат выполнения команды

Создайте индекс для ключа value.

db.numbers.createIndex({ value: 1 })

Получите информацию о всех индексах коллекции numbres.

db.numbers.getIndexes()

На рисунке 55 результат выполнения команд.



Рисунок 55 – Результат выполнения команды

Выполните запрос 2.

db.numbers.explain("executionStats").find().sort({ value: -1 }).limit(4)

На рисунке 56 результат выполнения команд.

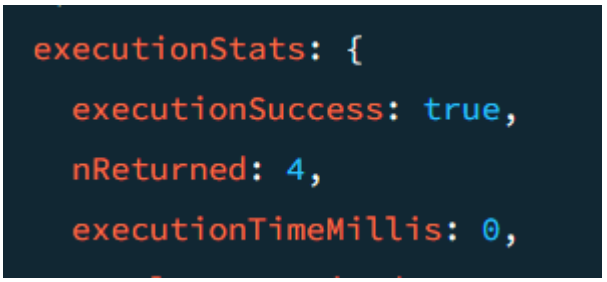


Рисунок 56 – Результат выполнения команды

Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?

Первый запрос: MongoDB просканировала все 100,000 документов, чтобы выполнить сортировку по value: -1 и вернуть 4 последних.

Второй запрос: Запрос мгновенно вернул 4 записи благодаря использованию индексированной сортировки.

Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

Первый запрос: 34 мс.

ВТорой запрос: 0 мс.

Запрос с индексом по полю value значительно эффективнее.

Он выполняется в десятки раз быстрее, не требует полной выборки, и не делает сортировку в памяти.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была достигнута основная цель — овладение практическими навыками работы с базой данных MongoDB. Были последовательно освоены операции создания базы данных и коллекций, вставки документов различными способами, работы с вложенными структурами и массивами, а также применение запросов с условиями, сортировкой и проекцией.

В процессе работы были реализованы следующие ключевые этапы:

- создание и удаление базы данных learn и коллекций unicorns, numbers, towns;
- вставка документов с использованием как простого insert, так и insertMany, а также автоматического связывания через DBRef;
- выполнение запросов с использованием фильтрации по полю, весу, предпочтениям, и проверке на наличие или отсутствие ключей;
- применение сортировки (sort), ограничения количества результатов (limit) и проекции (projection);
- работа с массивами, включая извлечение первого элемента через $slice;
- использование логических, сравнительных и специальных операторов: $exists, $all, $natural, $push, $unset, $inc;
- модификация и обновление отдельных документов и полей с помощью updateOne, updateMany, включая добавление и изменение вложенных массивов;
- подсчёт документов и агрегация с группировкой ($group, $unwind);
- создание, просмотр и удаление индексов, включая уникальные (unique) и составные индексы;
- анализ плана выполнения запросов через explain("executionStats") и оценка производительности запросов с и без индексов.

Также проведена оценка эффективности запросов при использовании индексов. Экспериментально подтверждено, что индекс по полю value значительно ускоряет выполнение запросов (сокращение времени выполнения с 34 мс до 0 мс), за счет исключения полного сканирования коллекции и сортировки в памяти.

Выводы:

Получены уверенные навыки работы с MongoDB: вставка, обновление, удаление, поиск, фильтрация, сортировка и агрегация данных.

Освоены методы работы с вложенными объектами, массивами и автоматическими связями между коллекциями через DBRef.

На практике подтверждена значимость использования индексов для повышения производительности.

Изучены и применены инструменты, необходимые для построения гибких и масштабируемых NoSQL-решений.